

Lab 1 :datalab

费浚洋 23300660004

```
merlin@Merlin:~/lab1$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 9/18 operators
secondLowestZeroBit: 5/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 14/24 operators
midpointTowardFirst: 23/32 operators
isBetweenEitherOrder: 29/48 operators
mul5Sat: 17/30 operators
classifyAdd3: 21/52 operators
floatScaleThreeHalves: 33/60 operators
floatRoundEven: 27/65 operators
float_i2f: 30/40 operators
bitCount: 24/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

```

merlin@Merlin:~/lab1$ ./btest
Score   Rating   Errors   Function
1       1         0       signMask
2       2         0       bitXor
3       3         0       negativePart
4       4         0       copyByteWithin
4       4         0       logicalShift
4       4         0       swapNibblePairs
4       4         0       secondLowestZeroBit
5       5         0       oddParity
5       5         0       rotateRightBits
5       5         0       roundEvenPow2
5       5         0       midpointTowardFirst
7       7         0       isBetweenEitherOrder
7       7         0       mul5Sat
7       7         0       classifyAdd3
7       7         0       floatScaleThreeHalve
10      10        0       floatRoundEven
10      10        0       float_i2f
10      10        0       bitCount
10      10        0       bitReverse
Total points: 110/110

```

P1 signMask

这题需要让最高位为 1，其他位为 0。int 一共有 32 位，所以把 1 左移 31 位就能得到结果。

P2 bitXor

这题只能用取反和按位与，不能直接用异或。我根据异或的特点，把两个位相同的情况排除掉。两个位都是 1 或者都是 0 时返回 0，只有两个位不同时返回 1，这样就实现了异或。

P3 negativePart

先把 x 右移 31 位，得到它的符号信息。负数会得到全 1，非负数会得到全 0。然后用取反加 1 求出 x 的相反数，再和前面的结果按位与。这样负数会保留取反后的结果，非负数则返回 0。

P4 copyByteWithin

先根据 src 找到要复制的字节，把它右移到最低字节的位置，再和 255 按位与，去掉其他位。之后把目标位置原来的字节清零，再把取出的字节左移到目标位置，合并回原数。这样只会改变目标字节。

P5 logicalShift

负数直接右移会在高位补 1，但逻辑右移要求补 0。所以我先正常右移，再用一个掩码把高位补出来的 1 清掉，只保留应该留下的低位。构造掩码时也考虑了不移动的情况，避免出现移位 32 位的问题。

P6 swapNibblePairs

这题要交换每个字节里的高 4 位和低 4 位。我先用掩码取出每个字节的低 4 位，再左移 4 位。另一部分先右移 4 位，再用掩码取出原来的高 4 位。最后把两部分合起来，就完成了交换。

P7 secondLowestZeroBit

我的思路是先把最低的一个 0 变成 1，这样原来第二低的 0 就变成了最低的 0。具体是先计算 x 和 x 加 1 的按位或，然后对这个结果取反，再与它加 1 的结果按位与，提取出对应的位置。如果原来的数不足两个 0，最后就会返回 0。

P8 oddParity

这题只需要知道 1 的个数是奇数还是偶数，不需要算出具体有多少个 1。我把 x 依次与自身右移 16 位、8 位、4 位、2 位和 1 位的结果异或，把所有位的奇偶信息集中到最低位。因为题目要求偶数个 1 时返回 1，所以最后还要把最低位的结果取反。

P9 rotateRightBits

循环右移可以分成两部分，一部分是正常右移后留下的位，另一部分是从低位移出去、需要补到高位位。我先把移动位数对 32 取模，再分别计算这两部分，最后按位或合并。右移部分要清除符号扩展，左移的位数也要处理好，保证不会移位 32 位。

P10 roundEvenPow2

我先看 x 除以 2 的 n 次方后，得到的商是奇数还是偶数。然后在 x 上加一个用于舍入的偏移量，最后把低 n 位清零。偏移量取半个间隔减 1，再加上商的最低位。这样不到一半时向下取，超过一半时向上取，刚好一半时就会取到商为偶数的那个结果。

P11 midpointTowardFirst

直接把 x 和 y 相加可能溢出，所以我用按位与、异或和右移来计算向下取整的平均值。如果两数之和是偶数，就直接返回这个结果。如果是奇数，就判断 x 是否大于 y 。 x 大于 y 时把结果加 1，否则保持不变，这样取整后就会更靠近第一个参数 x 。比较大小时也区分了同号和异号，避免溢出影响判断。

P12 isBetweenEitherOrder

我分别判断 x 是否小于 a 、是否小于 b 。如果两个判断的结果不同，就说明 x 处于两个端点之间，或者等于较小的端点。因为题目要求包含端点，所以再加上 x 等于 a 或 b 的判断。这样不需要提前确定 a 和 b 谁大谁小。

P13 mul5Sat

我先通过加法算出 x 的 2 倍、4 倍和 5 倍，然后检查这些结果有没有出现和 x 不同的符号。如果出现符号变化，就说明发生了溢出。正数溢出时返回最大的 `int`，负数溢出时返回最小的 `int`，没有溢出就返回 5 倍的结果。最后用掩码选择返回值，不使用条件语句。

P14 classifyAdd3

三个整数直接相加容易溢出，所以我把每个数拆成高位和最低两位，分别求和。低位相加产生的进位再加到高位和里，这样可以保留判断溢出所需的信息，同时避免计算过程本身溢出。最后检查高位结果是否超出允许范围，正向溢出返回 1，负向溢出返回负 1，其他情况返回 0。

P15 floatScaleThreeHalves

我先把浮点数拆成符号位、阶码和尾数。如果是 NaN 或无穷大，就直接返回原值。对于普通数，先补好有效数，再用整数运算算出它的 3 倍，通过右移和调整阶码实现乘以 1.5。右移时会检查被丢掉的位，超过一半就进位，正好一半就根据保留部分的奇偶性决定。最后处理可能出现的进位和溢出，再拼回浮点数的编码。

P16 floatRoundEven

这题先根据阶码判断数的大小。绝对值小于或等于 0.5 时舍入到 0，并保留原来的符号；大于 0.5 但小于 1 时舍入到正 1 或负 1。对于其他带小数的数，先确定哪些位属于小数部分，再判断它是小于一半、大于一半，还是正好一半。正好一半时选择偶数。已经是整数的数，以及 NaN 和无穷大，都直接返回原值。

P17 float_i2f

先记录 x 的符号，再用无符号数保存它的绝对值，这样最小的负整数也能正常处理。然后找到最高位的 1，根据它的位置计算阶码。如果有效位不超过 24 位，就直接移动到对应位置；如果超过 24 位，就需要舍去一部分低位，并按照最近偶数的规则舍入。如果舍入后又产生了进位，还要相应调整阶码。

P18 bitCount

我用分组相加的方法统计 1 的个数。先统计每两位中有几个 1，再把相邻的结果合并，得到每 4 位中 1 的个数，然后继续合并成每 8 位的结果。最后把各字节的计数加起来。整个过程中用掩码分隔各组，避免它们互相影响。

P19 bitReverse

我没有逐位处理，而是分几步交换位置。先交换相邻的位，再交换相邻的两位组，然后交换每个字节中的高 4 位和低 4 位。这样每个字节内部的位序就反过来了，最后再把四个字节的顺序反过来，就完成了整个整数的位反转。为了不超过 34 个操作，我复用了已有的掩码，通过移位和异或生成其他掩码，减少了操作次数。